The ϵ-Greedy Policy: Exploration vs. Exploitation 🧭

When the learning algorithm is running, the Neural Network's estimates of $Q(s, a)$ are initially random and likely wrong. This creates a dilemma: **How do we choose actions based on a "brain" that doesn't know anything yet?**

The Problem with Being "Greedy"

If we *always* choose the action with the highest Q-value (a "Greedy" approach):

1. Suppose the network is initialized such that "Fire Main Engine" has a very low value.
2. The agent will **never** choose "Fire Main Engine" because it thinks it is bad.
3. Because it never tries it, it never receives the positive reward for using it.
4. **Result:** The agent gets stuck in a loop of bad behavior and never discovers the optimal strategy.

The Solution: ϵ-Greedy

To solve this, we introduce randomness. We choose a parameter called **Epsilon (ϵ)**, which represents the probability of acting randomly.

- **Exploitation (Probability $1 - \epsilon$):**
  - We use what we have learned so far.
  - We pick the action that maximizes $Q(s, a)$.
  - *Example:* 95% of the time ($\epsilon = 0.05$).
- **Exploration (Probability ϵ):**
  - We ignore the neural network.
  - We pick an action **completely at random**.
  - *Example:* 5% of the time.

Why do this?
Random actions force the agent to try things it thinks are "bad." Occasionally, one of these random actions will result in a high reward, allowing the Neural Network to update its Q-values and "realize" that the action is actually good.

Exploration vs. Exploitation Trade-off

This is a fundamental concept in Reinforcement Learning.

- **Exploration:** Trying new things to gather more information (Risk of low reward now, potential for high reward later).

- **Exploitation:** Using current knowledge to maximize the immediate reward (High reward now, no new knowledge gained).

Refinement: Decaying Epsilon

In practice, we usually change ϵ over time.

1. **Start High ($\epsilon \approx 1.0$):** At the beginning, the agent knows nothing. We want it to act randomly to explore the environment thoroughly.
2. **Decrease Gradually:** As the agent learns, we slowly lower ϵ.
3. **End Low ($\epsilon \approx 0.01$):** Once the agent is trained, we want it to use its learned skills (Exploit) 99% of the time, keeping only a tiny chance of randomness to handle unexpected situations.

A Note on RL Difficulty

Prof. Ng notes that Reinforcement Learning algorithms are much more **"finicky"** than Supervised Learning algorithms.

- **Supervised Learning:** A bad learning rate might make training 3x slower.
- **Reinforcement Learning:** A bad parameter (like ϵ) might make training **10x or 100x slower**, or prevent learning entirely. Tuning these parameters is often the hardest part of RL.